SPM: Distributed Memory Systems

Background

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Interconnection Networks, some topologies
- A simple communication cost model
- Overlapping communication and computation
  - Synchronous and asynchronous communications

Interconnection Networks

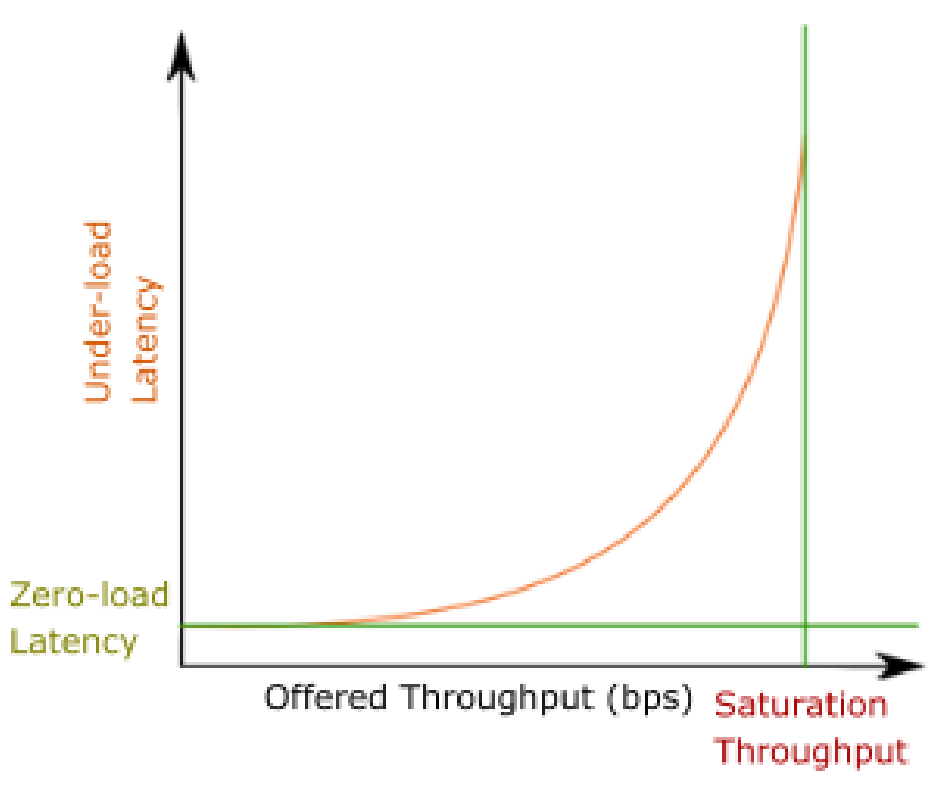
- HPC interconnection networks differ from WANs: shorter distances, controlled infrastructure, much stronger latency and bandwidth requirements
  - Usually, $O(10^1 \div 10^5)$ nodes connected to the same network infrastructure
  - Distances: from $O(10^0)$ to $O(10^1)$ meters
- Goal: low message latency, high bandwidth, acceptable cost
  - Usually, the cost is relatively high depending on the scale and the performance of the network
- Performance depends not only on hardware, but also on routing, flow control, and traffic contention
  - Routing: chooses the path followed by packets.
  - Flow control: regulates how much traffic can be injected into the network.
    - We will not study routing algorithms and flow control in detail

Network Performance Metrics

- **Latency**: is the time from when a packet starts to be transmitted from the source node and when it is completely received at the destination node. Expressed in time units: sec, ms, us, ns
  - **No-load (or zero-load) latency**. The time measured in a network when there is no traffic (thus no congestion). It represents the baseline performance of the network
  - **Under-load latency**. The time measured when the network is utilized, and the traffic is below the saturation point (i.e., the load is below the network capacity)
    - Over-load latency. Latency measured when the network is congested (above the saturation point)
- **(Offered) Throughput**: the actual amount of data sent into the network per unit of time. Expressed in bits per second (bps) or multiples (Kb/s, Mb/s, Gb/s)
  - **Saturation Throughput:** the maximum amount of traffic sustained by the network. It is the point at which the network is fully utilized
- **Bandwidth:** the theoretical maximum data transfer rate under ideal conditions across a given network path. It represents the max capacity of a communication channel. Expressed in Mb/s, Gb/s

Network Performance Metrics

- **Latency is not constant**
  - It depends on network contention and the distance between the source and the destination node
- As offered throughput approaches the saturation throughput, latency raises sharply
- The side figure sketches the general behavior of latency vs. offered throughput (i.e., injection rates). As the nodes keep injecting traffic into the network, it reaches a saturation point where the latency grows fast due to contention



Basic network terminology

- **Endpoints**: They are the sources and destinations of messages
  - We consider endpoints the computing nodes. In a distributed system, a computing node is equipped with a NIC (Network Interface Card) that sends data to and receives data from the network on behalf of the processor
- **Switches**: A switch is a device connected to a set of links. It transmits received packets to one or multiple links using routing logic to manage multiple simultaneous flows
- **Links**: It is a physical connection (a *wire*) used to transfer data between endpoints, endpoints, and switches, and between switches
  - High-speed serial connections (fiber optics, copper, …)

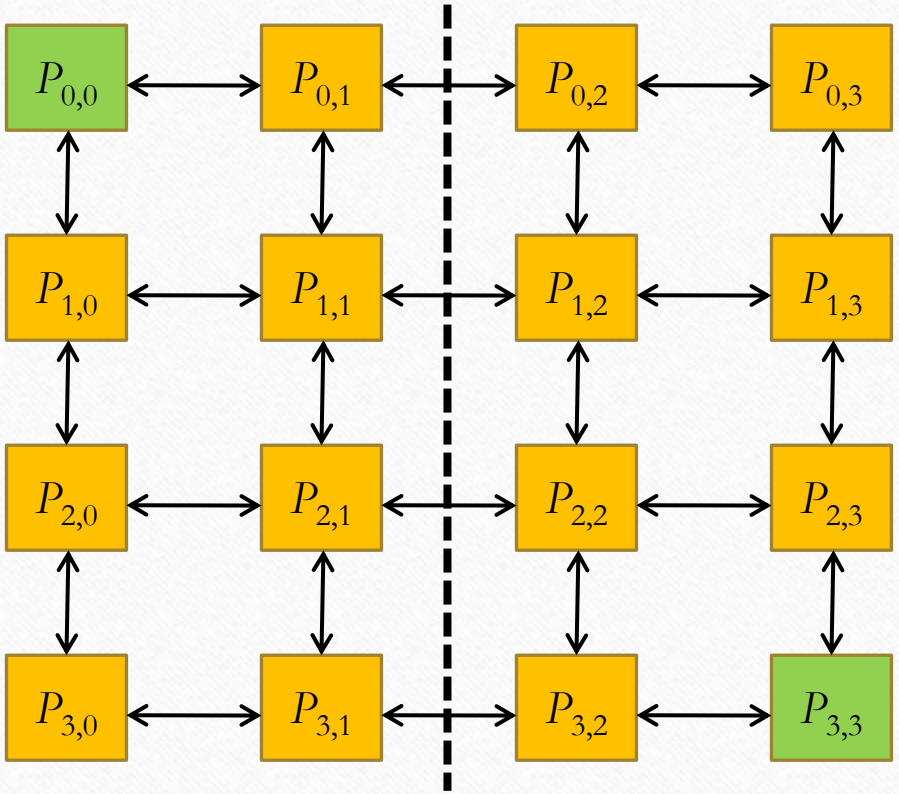
Direct and Indirect Networks

- **Direct networks**: In a direct network, the nodes are both endpoints and switches
  - Nodes also route traffic
  - Also called ***static networks***
  - Examples are: ring, nD-mesh, hypercube
- **Indirect networks**: In an indirect network, the endpoints are connected indirectly through switches
  - Dedicated switches route traffic
  - Also called ***dynamic*** *or* ***multistage networks***
  - Examples are: Butterfly, Fat Tree, Dragonfly

Diameter, Degree and Bisection-Width

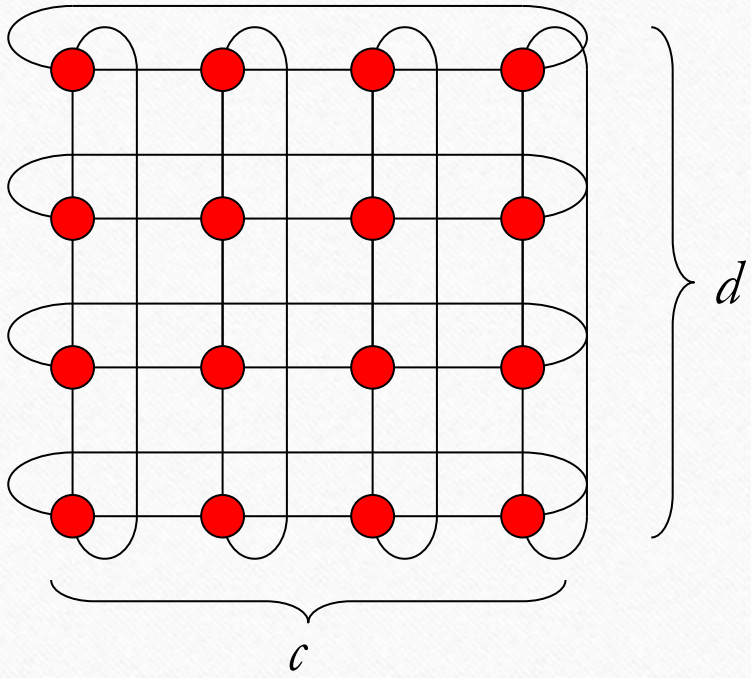
- **Degree**: The degree (*deg*) of a network is the maximum number of neighbors of any node
- **Diameter**: The diameter (*diam*) of a network is the length of the longest of all shortest paths between any two nodes
  - Length measured in hops or edges along the shortest path
  - The diameter has an impact on the worst-case latency
- **Bisection-width**: The bisection-width (*bw*) of a network is the smallest number of edges (or links) to be removed to disconnect the network into two halves of (nearly) equal size
  - In case of an odd number of nodes, one of the two halves can include one more node
  - It relates to how well a network can handle bulk data movement across two large groups of nodes

2D Mesh



- $M(d,d)$ has $n = d^2$ nodes (endpoints)
- $deg(M(d,d)) = 4$
  - $\boldsymbol{O(1)}$
- $dia(M(d,d)) = 2(d-1)$
  - $\boldsymbol{O(\sqrt{n})}$
- $bw(M(d,d)) = d$
  - $\boldsymbol{O(\sqrt{n})}$

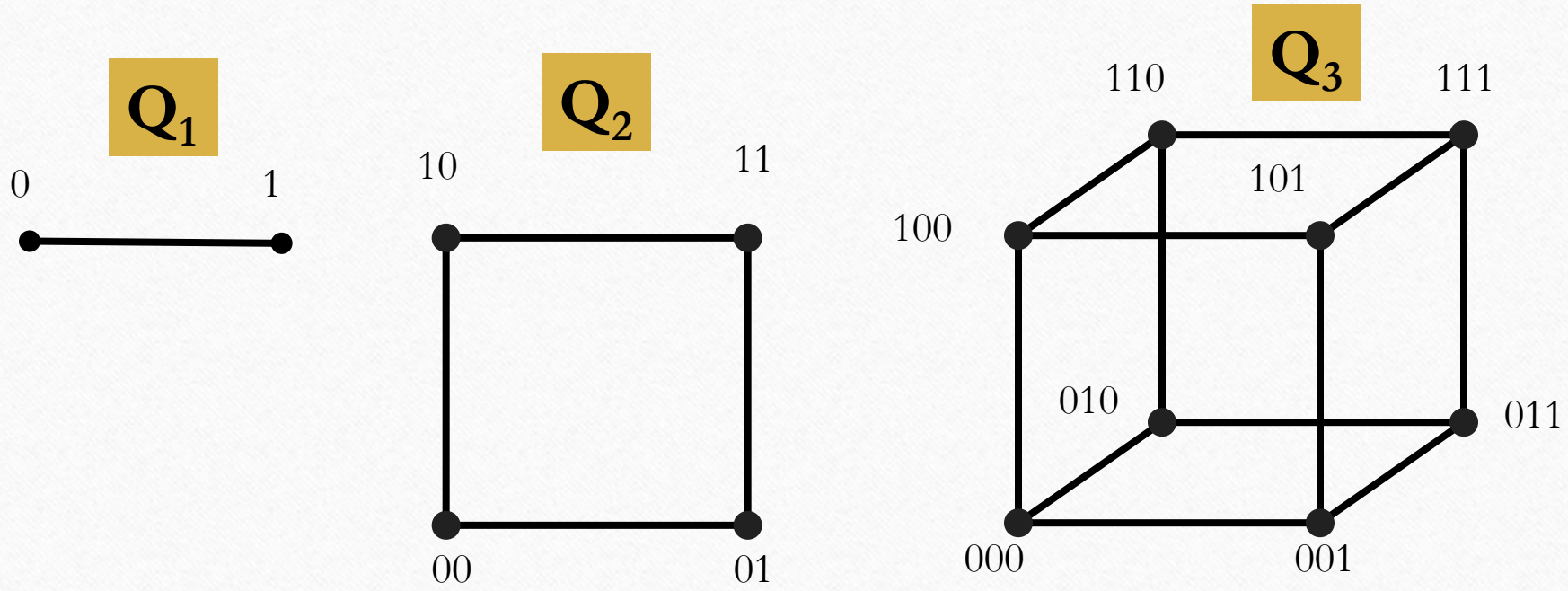
2D Torus



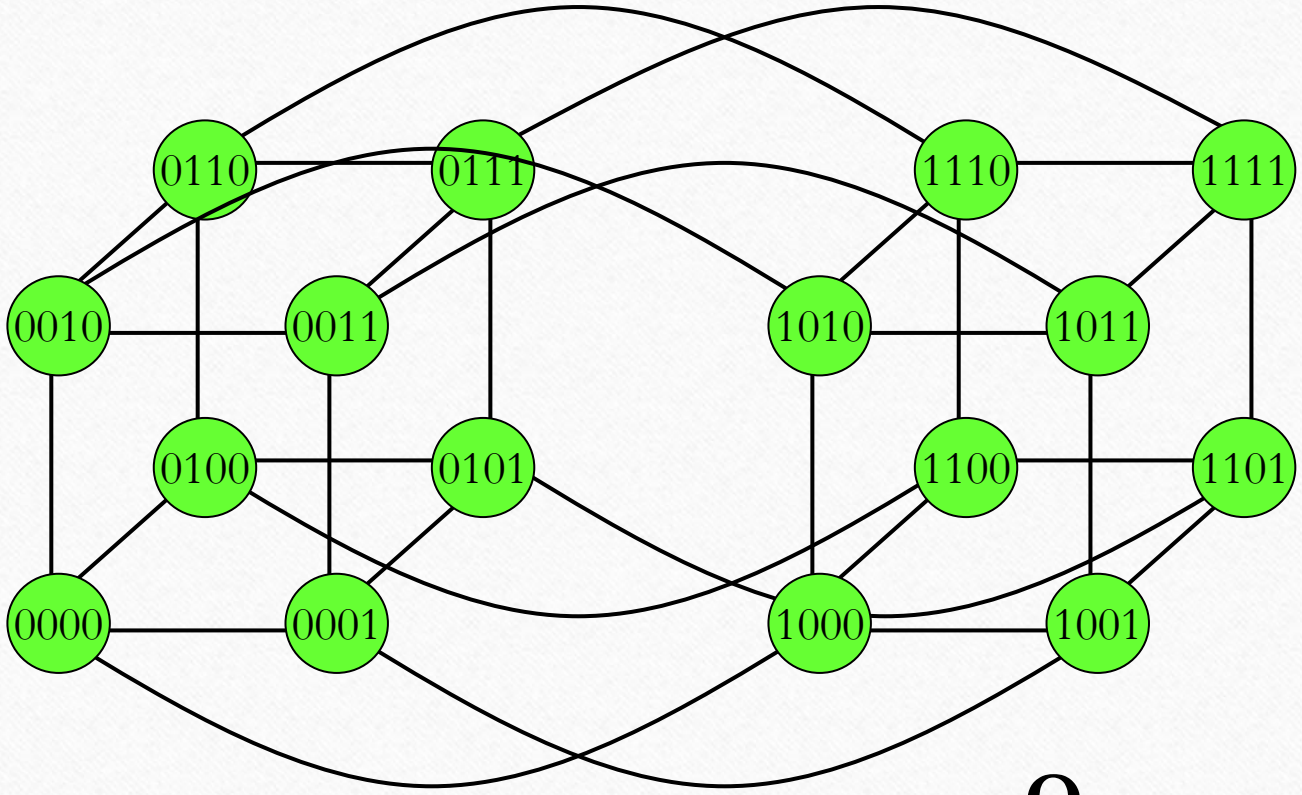
- A torus $T(c,d)$ is a Mesh augmented by wraparound edges at the border of the mesh.
- $T(c,d)$ has $\boldsymbol{n = c \cdot d}$ nodes/endpoints
- $deg(T(c,d)) = 4$
  - $\boldsymbol{O(1)}$
- $dia(T(c,d)) = \left\lfloor \frac{d}{2} \right\rfloor + \left\lfloor \frac{c}{2} \right\rfloor$
  - $\boldsymbol{O(\sqrt{n})}$
- $bw(T(c,d)) = \min\{2 \cdot c, 2 \cdot d\}$
  - $\boldsymbol{O(\sqrt{n})}$

Hypercube

The ***Hypercube***, denoted by Q_d ($d \geq 1$), is the graph that has vertices representing the 2^d bit strings of length d. Two vertices are adjacent if and only if the bit strings that they represent differ in exactly one bit position.



Hypercube



Q_4

- HC(d) = d-dimensional hypercube
  - Number of nodes: $n = 2^d$
  - $deg(HC(d)) = d$
    - $\boldsymbol{O(\log(n))}$
  - $dia(\mathrm{HC}(d)) = d$
    - $\boldsymbol{O(\log(n))}$
  - $\mathrm{bw}(HC(d)) = n/2$
    - $\boldsymbol{O(n)}$

Criteria for Evaluating Network Topologies

- ***Low*** **Diameter** to support efficient communication between any pair of processors
  - Reduces worst-case latency
- ***High*** **Bisection Width.** A low bisection width can slow down many *collective communication operations* (operations that involve a set of nodes)
  - However, achieving high bisection width may require a non-constant network degree
- ***Constant*** **degree** (i.e. independent of network size)
  - allows a network to scale to a large number of nodes without the need to add an excessive number of connections
  - Keeps physical cost scalable

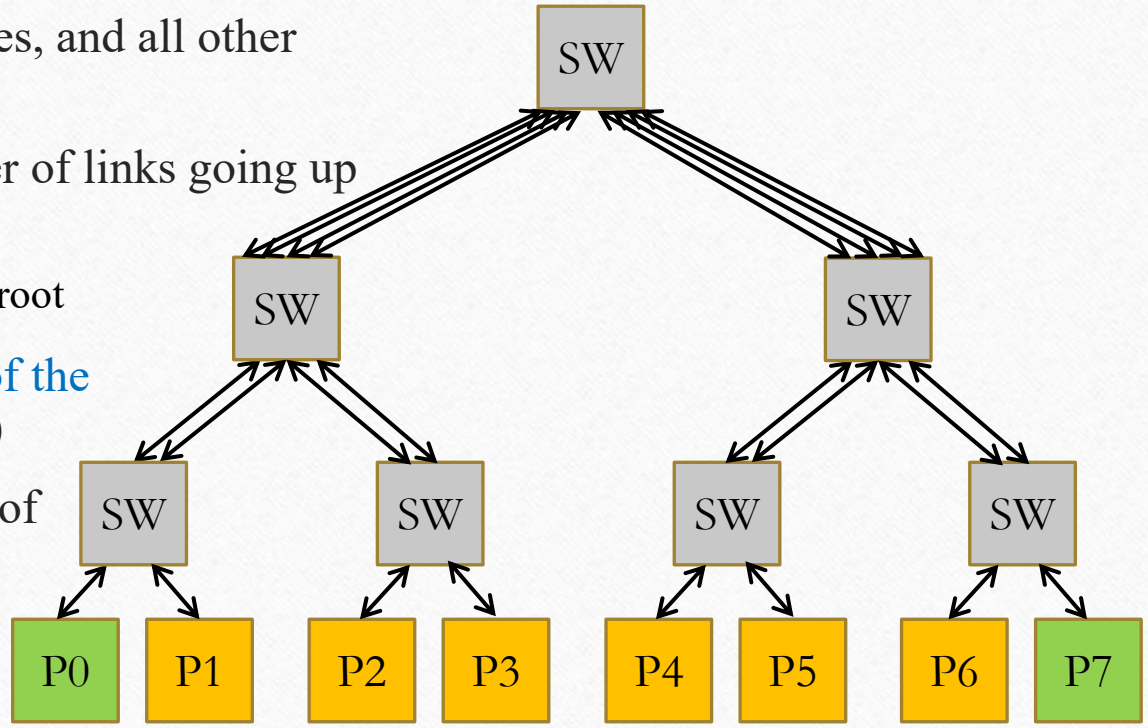
Topology	Degree	Diameter	Bisection Width
Linear Array	$O(1)$	$O(n)$	$O(1)$
2D Mesh/Torus	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$
3D Mesh/Torus	$O(1)$	$O(\sqrt[3]{n})$	$O(n^{2/3})$
Binary Tree	$O(1)$	$O(\log(n))$	$O(1)$
Hypercube	$O(\log(n))$	$O(\log(n))$	$O(n)$

No topology optimizes all three metrics at the same time

Fat Tree

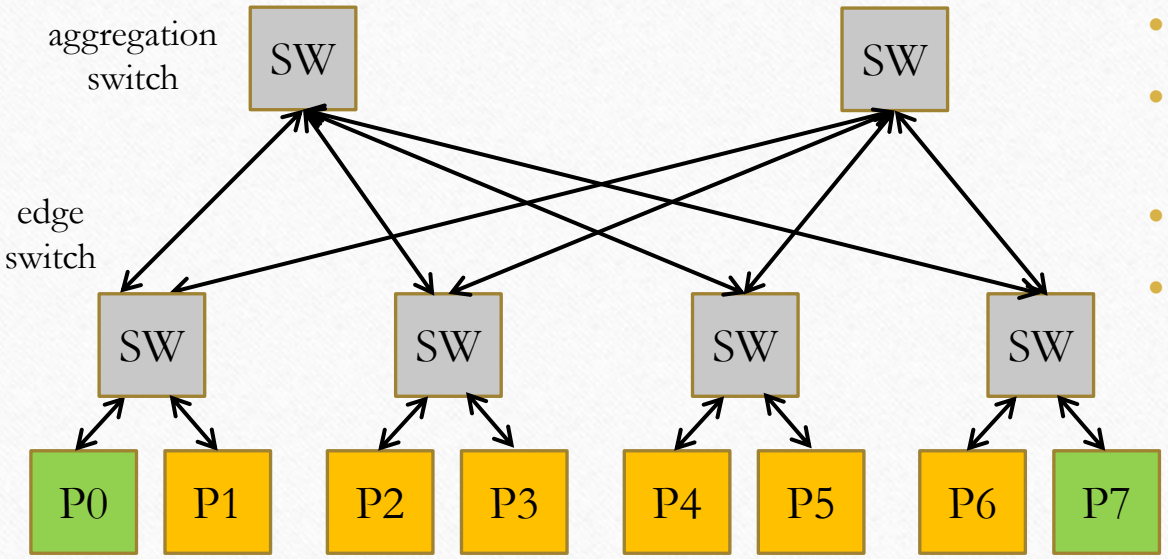
1/2

- Indirect tree topology in which the endpoints are the leaves, and all other nodes are switches
- The characteristic is that each switch has the same number of links going up and going down
  - The number of links increases (getting fatter) towards the root
- The idea is to keep the bandwidth constant at each level of the tree (i.e., we have the same number of links at each level)
- The main problem is the cost! It increases with the depth of the tree.
  - Top-level switches have too many links
  - Not realistic as sketched in the picture



Fat Tree 2/2

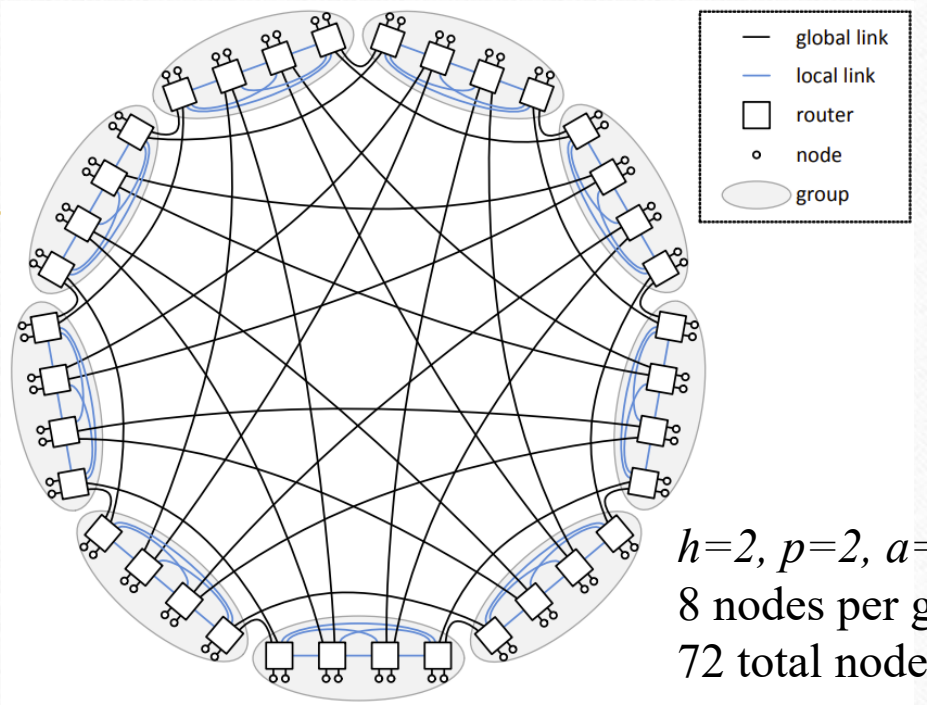
- Implemented in a different way, using switches with a **limited degree**, i.e., each switch has at most *k* ports, and organized in a multi-stage network (*k-ary n-fly* network and *folded clos* network topologies)



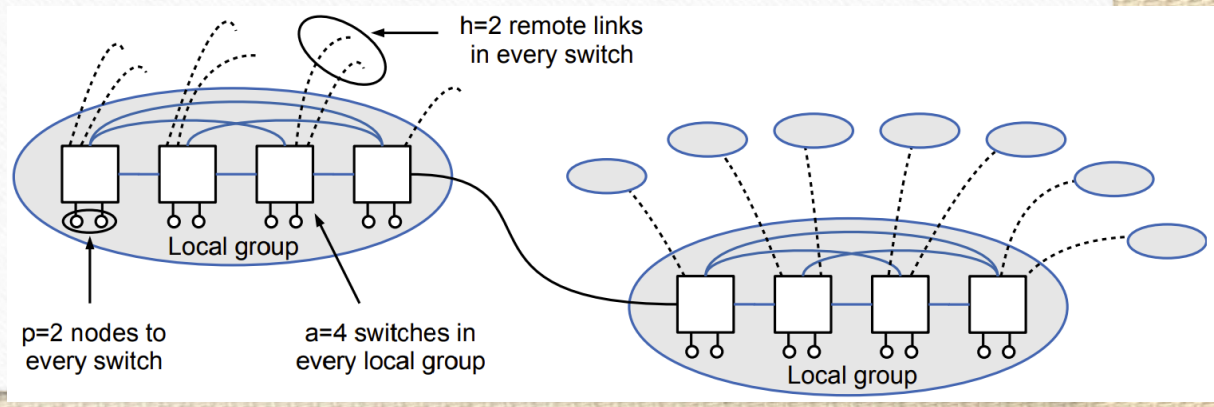
- In the figure, a 2-Level Fat Tree (i.e., 2 levels of switches)
- Each switch has the same number of ports (k=4 in the picture)
- Low-cost solution, limited by the number of ports of the switch
- To scale out the network we can increase the number of levels
- For a 2-level Fat Tree where each switch has *k* ports:
  - $k + \frac{k}{2}$ switches in total
  - $n = k \times \frac{k}{2}$ maximum number of endpoints
  - bw(n) = $\frac{n}{2}$, deg(n)=k, dia(n)=4

Dragonfly

- **Multi-level indirect network**: routers are organized into groups
- Each router has connections to ***p*** endpoints, ***a-1*** local channels (to other routers in the same group) and ***h*** global channels (to routers in other groups)
  - *deg(DF(p,a,h)) = p + (a-1) + h*
- A group consists of ***a*** routers. Each group has ***ap*** connections to endpoints (i.e., fully connected topology) and ***ah*** connections to global channels
- Dragonfly networks aim at low constant diameter and high bisection bandwidth, but their performance depends critically on non-trivial routing and traffic balancing across global links

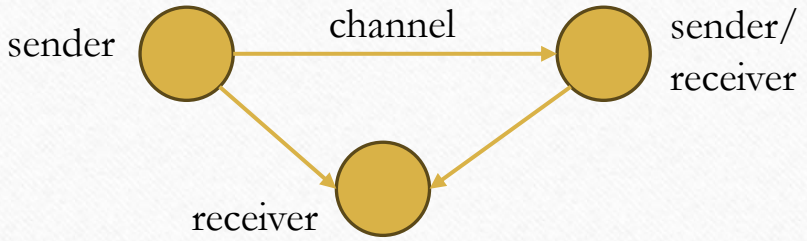


h=2, p=2, a=4
8 nodes per group
72 total nodes



Message-Passing Semantics 1/2

- **Message-Passing** is a computational paradigm where independent processes or agents communicate solely by sending and receiving messages
  - No physical or logical shared memory among communicating entities
  - Each process has its local state, sharing implemented through messaging
- It abstracts the communication aspects of concurrent and distributed systems, enforcing isolation making failure handling and scaling simpler
- Theoretical models and formalisms:
  - Actor Model: Processes (actors) encapsulate state and behavior, interacting through asynchronous messages. Each actor has a mailbox and processes messages sequentially
  - Process Calculi: CSP (Communicating Sequential Processes) and the π-calculus, which provide formal frameworks to describe and analyze interactions among concurrent entities



Message-Passing Semantics 2/2

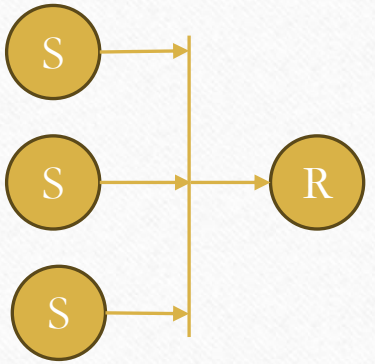
- Key aspects of the Message-Passing model are:
  - Asynchrony/synchrony between sender and receiver
    - synchronous: coupled send/receive; asynchronous: non-blocking send, potentially blocking receive
  - Input non-determinism
    - Message arrival order may vary
  - Ordering/Causality
    - There is no global clock or global ordering of events. Processes observe events in a partial order
- Scalability: Message-Passing naturally supports distributed systems (microservices, serverless computing, HPC)
- Challenges: deadlock (circular waiting of messages) and livelock (no real progress)
- Generality: can also be used in a shared-memory system (e.g., Go channels)
  - The implementation of the channel can be a concurrent queue

Synchronous vs. Asynchronous communications

- The **asynchrony degree** of a channel (or channel capacity) is the maximum number of messages ($k \geq 0$) the sender can send before it has to block waiting for the receiver to start receiving data
  - It depends on the memory capacity of the channel and the size of the message being sent
- **Synchronous communications**: A send/receive operation is called synchronous if the operation completes only after the message has been received/sent
  - sender-receiver **rendezvous**. The sender/receiver blocks until the communication peer completes the operation
- **Asynchronous communications**: The communication operation returns immediately without waiting for the message to be actually sent/received
  - The completion/success of the communication will be tested later on (e.g., callbacks, futures/promises)
  - The number of asynchronous sends might be limited by the asynchrony degree of the channel. Even asynchronous communication can lead to blocking if the channel buffer is finite and full

Input non-determinism

- When a logical input channel multiplexes a set of independent channels, input non-determinism refers to the property that the *receive* operation may return a message coming from any channel in the set
  - There is no fixed order when receiving a message from a multi-input channel
- If more than one message is ready at once, one is chosen arbitrarily (i.e., non-predictable order)
- Fixing a receiving order is usually a non-optimal approach as it may reduce flexibility and add overhead
  - If strict ordering is needed for correctness, an additional layer implementing ordering protocol is often implemented on top of the non-deterministic messaging layer



A simple communication cost model 1/2

- Simplified model. Enough to reason about the cost of communication in parallel applications
- The data transfer time (or communication time) can be described by a simple **linear model**:

$$T_{comm} = t_0 + n \cdot s \approx \begin{cases} t_0 & \text{for small n} \\ n \cdot s & \text{for large n} \end{cases}$$

  - t_0 is the communication startup overhead (network and communication runtime setup)
  - n is the amount of data to be transferred (i.e., number of bytes)
  - s is the transmission cost (per-byte cost)

A simple communication cost model 2/2

$$T_{comm} = t_0 + n \cdot s \approx \begin{cases} t_0 & \text{for small n} \\ n \cdot s & \text{for large n} \end{cases}$$

- t_0 also called «latency»
  - It contains all costs for sending the shortest message (e.g., 1-byte payload, i.e., n=1).
  - Its value may be different for different message-passing implementations (e.g., because of data copies, or sync vs async operations)
- s usually estimated with $s = \frac{1}{B}$ where B is the available bandwidth along the transmission path
  - It may include also all the cost associated to the size of the message (e.g., message copies)
  - s is limited by the slowest part of the path between the sender and receiver processes
  - s includes both SW contributions (i.e., the rate at which the application can feed the network) and HW contribution (i.e., the rate at which data moves along the wires, and switches of the network)
- On modern HPC systems, t_0 is about 1-10 usec whereas B is 100-200 Gb/s

Linear communication model pitfalls

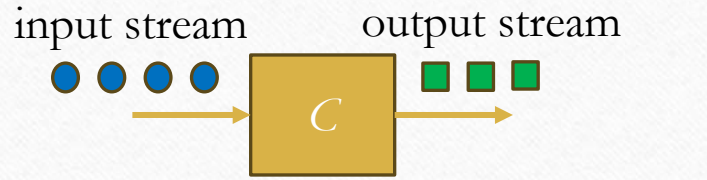
- The linear model **is simple**, and it applies to many fields of computer architectures
  - e.g., to model memory access time, bus transactions, pipeline operations
- However, **it has some limitations**
  - Does not include the effect of network distance (network hops)
  - Does not model contention (that latency depends on network utilization!)
    - Due to high network utilizations (many other messages in the network) and limited buffering in the network switches
  - Does not model how the data transfer is performed (i.e., synchronously or asynchronously)
- However, the goal is not to estimate T_{comm} precisely, but to hide or reduce its cost (e.g., by overlapping communication with computation)

Computation-communication overlap

- A computing node is equipped with a **NIC (Network Interface Card)** that sends data to and receives data from the network on behalf of the processor
- The NIC can be a **specialized processor** (so-called **SmartNIC**) with multiple capabilities other than simple DMA (also processing, e.g., compression)
- The sender process may execute a **non-blocking send** to the destination process, the NIC executes the data transfer and then notifies the sender process about the completion of the transmission
- While the NIC executes the data transfer, the processor does some other useful operations (e.g., it can execute, or partially execute, another task)
- There could be full or partial overlap between the computation of tasks and the communication with other processes. **Such overlap is fundamental to mask (or partially mask) communication overhead**

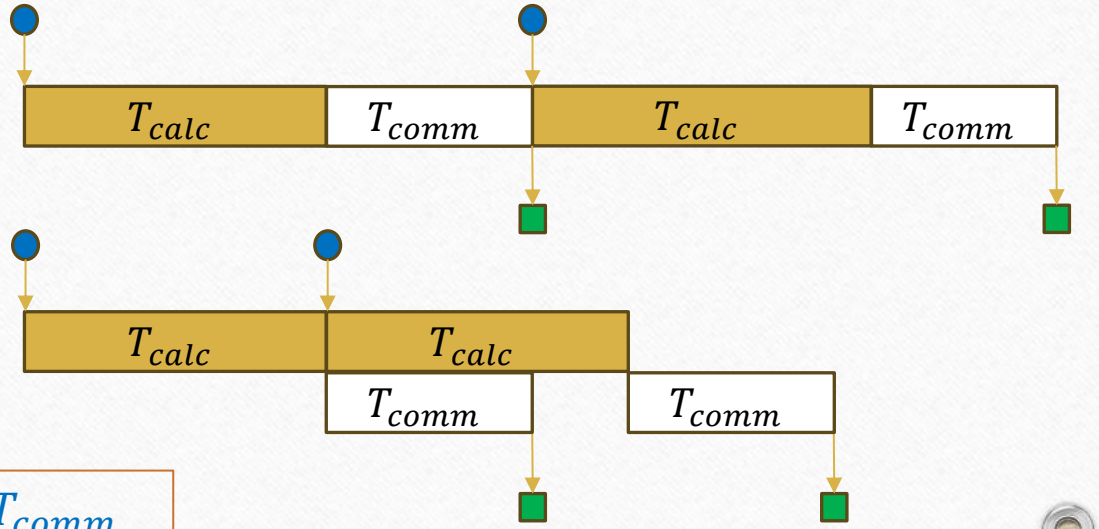
Computation-communication overlap

- The computing module *C* receives in input a list of tasks (***data stream***). For each input, it takes $\boldsymbol{T_{calc}}$ time for the computation and $\boldsymbol{T_{comm}}$ time for sending the computed task to the next module
- The **service time of a module** *C* (T_s^C) is the time interval between the start of processing two consecutive tasks
- We may have:
  - $T_s^C = T_{calc} + T_{comm}$ with **no** overlap
  - $T_s^C = \max(T_{calc}, T_{comm})$ with overlap



receive(task); compute(task); send(task)

Timing diagrams:



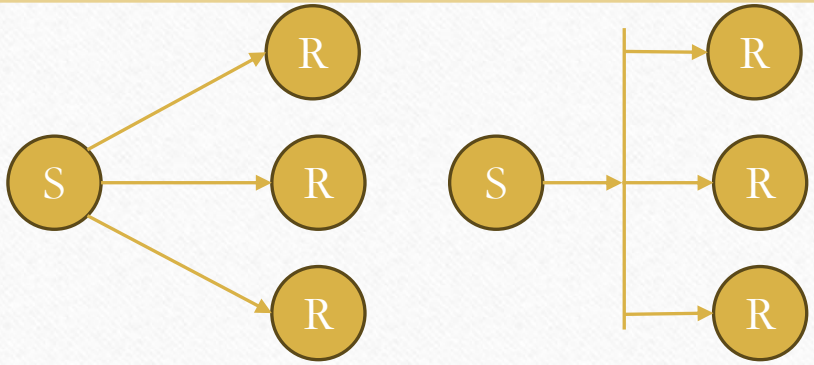
$$\max(T_{calc}, T_{comm}) \leq T_s^C \leq T_{calc} + T_{comm}$$

Computation-communication overlap

- SmartNIC may include RDMA (Remote Direct Memory Access) transfer. It enables memory-to-memory data transfer between two nodes without the continuous involvement of the operating system
  - InfiniBand provides hardware-level support for RDMA with dedicated switch fabrics
  - ROCE (RDMA-over Converged Ethernet) implements RDMA on top of Ethernet, allowing the same low-latency advantages in data-center environments without requiring InfiniBand infrastructure
- Hiding communication latency with RDMA. CPU offloads data transfer to the NIC and continues the processing
  - Bypasses CPU to enable direct memory read/write operations to/from remote nodes
  - Offers zero-copy, kernel-bypass data transfer
  - Common in high-performance networks (e.g., Infiniband)
- RDMA API (e.g., libibverbs) are generally more complex than standard API (e.g., sockets)
- Some Message-Passing libraries (e.g., MPI) leverage RDMA under the hood for fast point-to-point data transfer

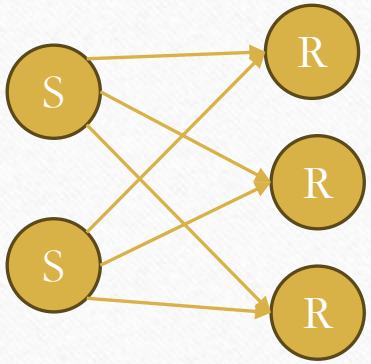
Communication patterns

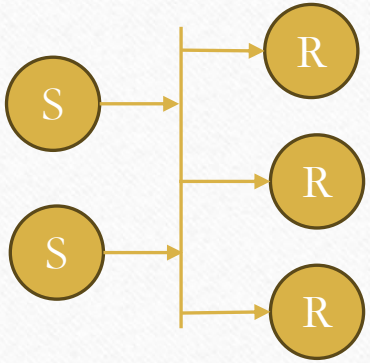
One-to-One



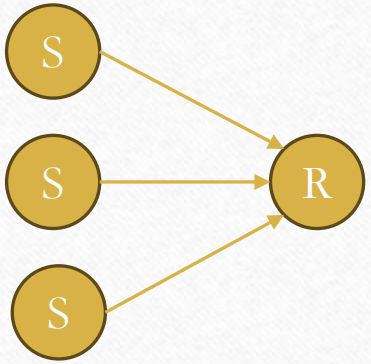
One-to-Many

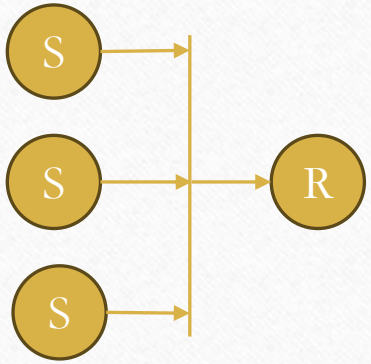
- Different channel semantics and implementations are useful for different communication patterns
- Communication cost also depends on the pattern, since each pattern stresses the network in different ways





Many-to-Many

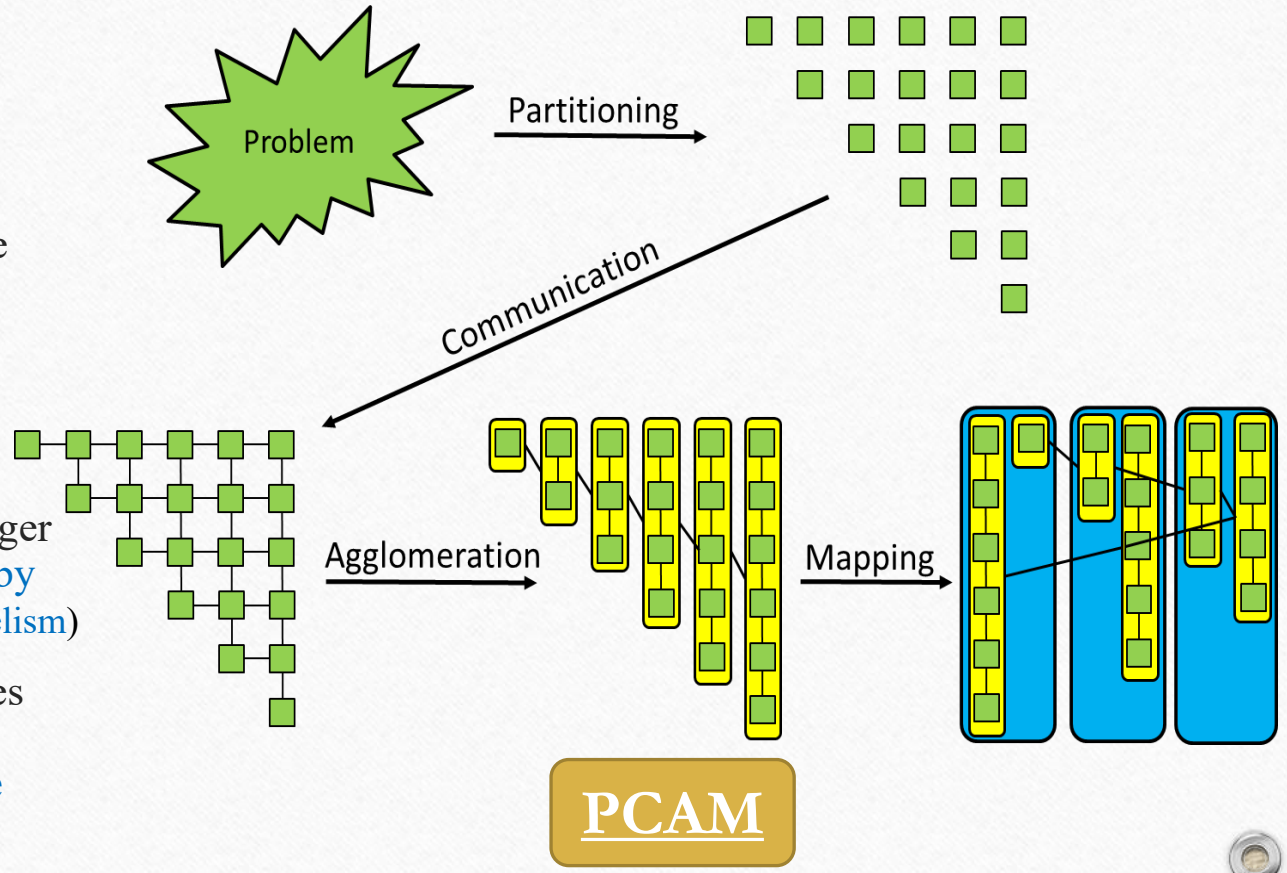




Many-to-One

Foster's Parallel Algorithm Design Method

- Conceptual framework for reasoning about parallelization of a given problem
- Ian Foster proposed the **PCAM** approach
  - **Partitioning:** decompose the problem into a large amount of small (*fine-grained*) tasks that can be executed in parallel.
  - **Communication:** determine the required communication between tasks (**dependencies**)
  - **Agglomeration**: combine identified tasks into larger (*coarse-grained*) tasks to reduce communication by improving data locality (balance overhead vs. parallelism)
  - **Mapping:** assign the aggregated tasks to processes according to the network topology to minimize communication, enable concurrency, and **balance workload**



Example: Jacobi Iteration

- **Stencil code** applied on a 2-dimensional array. Used to solve 2D PDE
- Update each value in the matrix with the average of its four neighbors

$$\text{data}_{t+1}(i,j) \leftarrow \frac{\text{data}_t(i-1,j) + \text{data}_t(i+1,j) + \text{data}_t(i,j-1) + \text{data}_t(i,j+1)}{4}$$

- The update rule is applied iteratively until convergence
- Boundary values (yellow part) remain constant at each iteration (fixed boundary conditions)
- At the end of each iteration, swap the updated array with the original one to avoid overwriting during the next iteration

The initial matrix

−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1
3	3	3	3	3	3	3	3	3	3	3	3
3	3	3	3	3	3	3	3	3	3	3	3
2	2	2	2	2	2	2	2	2	2	2	2
2	2	2	2	2	2	2	2	2	2	2	2
2	2	2	2	2	2	2	2	2	2	2	2
1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1

Example: Jacobi Iteration

- Replaces all points of a given 2D matrix by the average of the values around it in every iteration step until convergence:

```
copy(buff, data, rows, cols);
for (int k=1; k<MaxIter; k++) {
  for (int i=1; i<rows-1; i++)
    for (int j=1; j<cols-1; j++)
      buff[i*cols+j] = 0.25f * ( data[(i+1)*cols+j] + data[i*cols+j-1]
                        + data[i*cols+j+1]+data[(i-1)*cols+j] );
  residual = R(buff, data);  // e.g., L2-norm of the difference among cells
  if (residual < THRESHOLD) break;
  swap(data, buff, rows, cols);
}
```

- Boundary values are fixed:
  data[0][*], data[n-1][*], data[*][0], data[*][n-1]

The matrix after 1 iteration

−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1
3	2	2	2	2	2	2	2	2	2	2	3
3	2.8	2.8	2.8	2.8	2.8	2.8	2.8	2.8	2.8	2.8	3
2	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2
2	2	2	2	2	2	2	2	2	2	2	2
2	1.8	1.8	1.8	1.8	1.8	1.8	1.8	1.8	1.8	1.8	2
1	1.3	1.3	1.3	1.3	1.3	1.3	1.3	1.3	1.3	1.3	1
1	1	1	1	1	1	1	1	1	1	1	1
1	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	1
0	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0
0	-0.3	-0.3	-0.3	-0.3	-0.3	-0.3	-0.3	-0.3	-0.3	-0.3	0
−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1

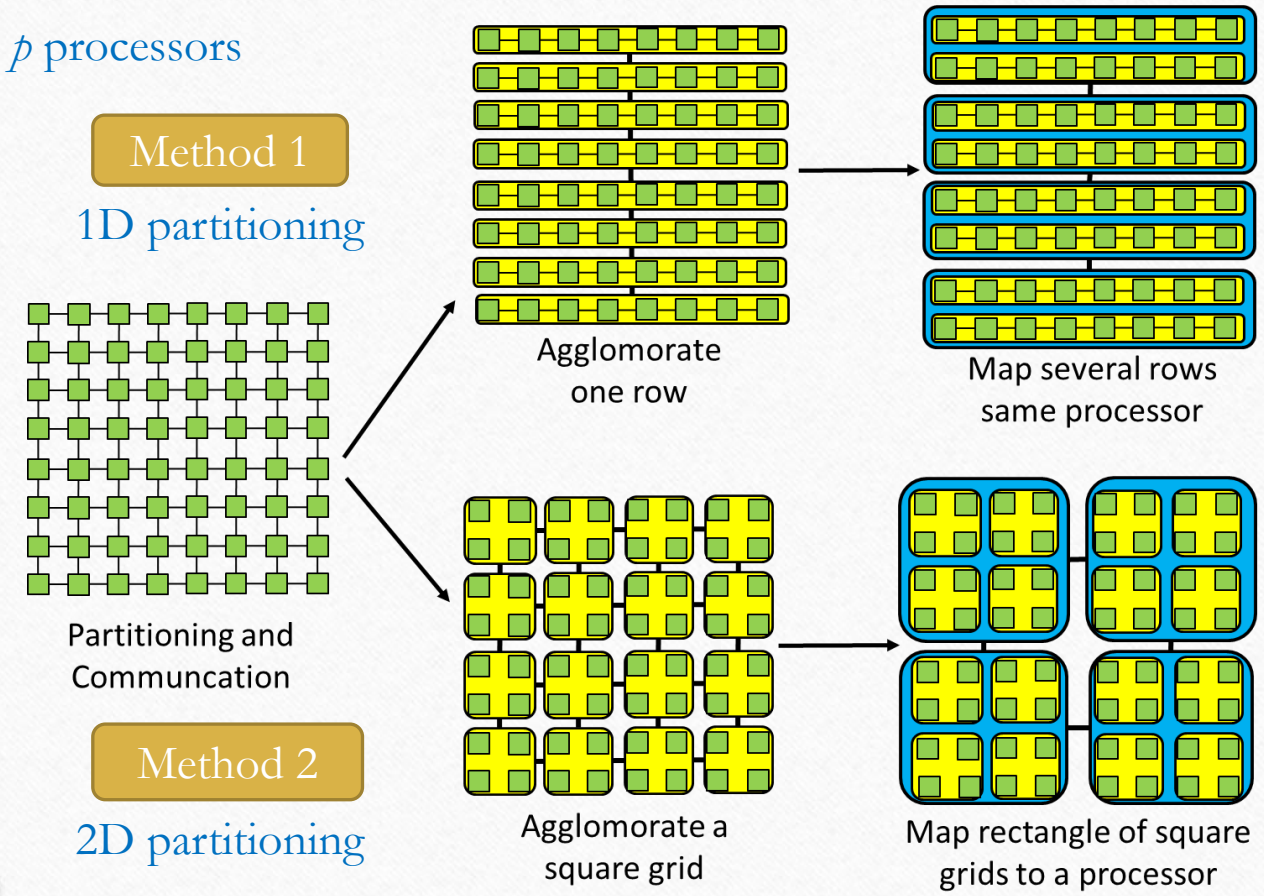
Example: Jacobi Iteration

The matrix after 25 and 75 iterations

−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1
3	1.0	0.2	−0.1	−0.3	−0.3	−0.3	−0.3	−0.1	0.2	1.0	3
3	1.6	0.9	0.5	0.4	0.3	0.3	0.4	0.5	0.9	1.6	3
2	1.6	1.2	0.9	0.8	0.7	0.7	0.8	0.9	1.2	1.6	2
2	1.6	1.3	1.1	1.0	0.9	0.9	1.0	1.1	1.3	1.6	2
2	1.5	1.3	1.1	1.0	1.0	1.0	1.0	1.1	1.3	1.5	2
1	1.1	1.0	1.0	0.9	0.9	0.9	0.9	1.0	1.0	1.1	1
1	0.9	0.8	0.7	0.7	0.7	0.7	0.7	0.7	0.8	0.9	1
1	0.6	0.5	0.4	0.3	0.3	0.3	0.3	0.4	0.5	0.6	1
0	0.1	0.0	0.0	−0.1	−0.1	−0.1	−0.1	0.0	0.0	0.1	0
0	−0.3	−0.5	−0.5	−0.5	−0.5	−0.5	−0.5	−0.5	−0.5	−0.3	0
−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1

−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1
3	0.9	0.1	−0.3	−0.4	−0.5	−0.5	−0.4	−0.3	0.1	0.9	3
3	1.5	0.7	0.3	0.1	0.0	0.0	0.1	0.3	0.7	1.5	3
2	1.5	1.0	0.6	0.4	0.3	0.3	0.4	0.6	1.0	1.5	2
2	1.5	1.1	0.8	0.6	0.5	0.5	0.6	0.8	1.1	1.5	2
2	1.4	1.0	0.8	0.6	0.5	0.5	0.6	0.8	1.0	1.4	2
1	1.0	0.8	0.6	0.5	0.5	0.5	0.5	0.6	0.8	1.0	1
1	0.8	0.6	0.4	0.4	0.3	0.3	0.4	0.4	0.6	0.8	1
1	0.5	0.3	0.2	0.1	0.1	0.1	0.1	0.2	0.3	0.5	1
0	0.0	−0.1	−0.2	−0.2	−0.3	−0.3	−0.2	−0.2	−0.1	0.0	0
0	−0.4	−0.5	−0.6	−0.6	−0.6	−0.6	−0.6	−0.6	−0.5	−0.4	0
−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1	−1

Two parallel schemes for Jacobi Iteration



- **Partitioning:** The smallest task is the computation of a single element of the Jacobi matrix
- **Communication:** Within an iteration all fine-grain tasks can be computed independently. Each task needs the data of four neighbors. At the end of each iteration, there is a synchronization barrier among all p processors, and data is exchanged.
- **Agglomeration**: Two options proposed: 1) by row (or by column); 2) by using a square grid.
- **Mapping:** it follows the policy used for the agglomeration to map coarse-grain tasks to processors. By row, contiguous groups of rows are assigned to the p processors; or by square grids, rectangles of square grids are assigned to the p processors organized in a $\sqrt{p} \times \sqrt{p}$ grid.

Two parallel schemes for Jacobi Iteration

- Problem size (grid size) $n \times n$; p processes running on p nodes
- Considering the **linear model** for the cost of point-to-point communications :

$$\boldsymbol{T_{comm}(n) = t_0 + n \cdot s}$$

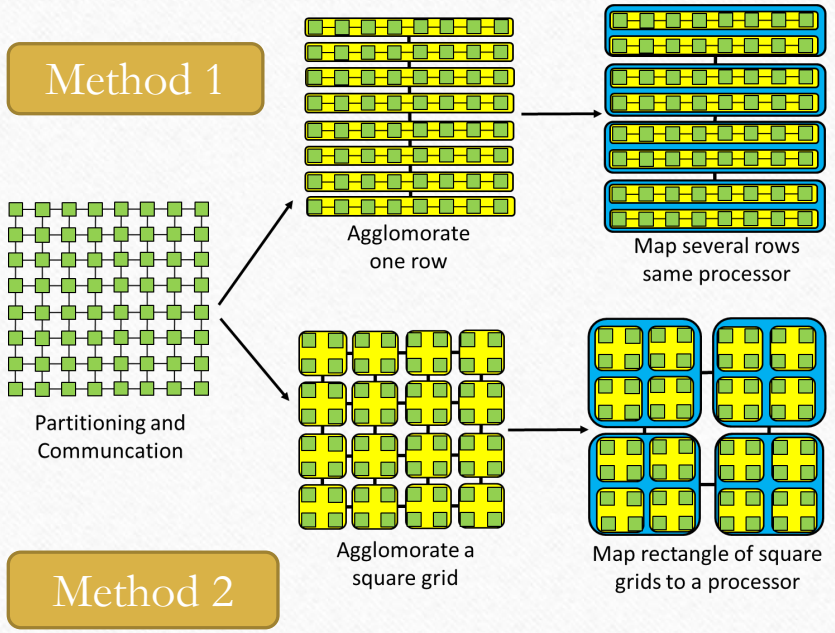
- **Method 1**: each process owns roughly $\frac{n}{p}$ rows or columns

$$T_{comm}(n) \approx 2 \cdot (t_0 + n \cdot s)$$

- **Method 2**: each process owns a $\frac{n}{\sqrt{p}} \times \frac{n}{\sqrt{p}}$ sub-block

$$T_{comm}(n) \approx 4 \cdot \left(t_0 + \left(\frac{n}{\sqrt{p}}\right) \cdot s\right)$$

Assume:
$p = \sqrt{p} \times \sqrt{p}$
(in the picture *p=4*)



- $\Rightarrow$ **Method 2** is preferable for large grids and sufficiently large p: lower communicated data per process, but more messages.

Suggested Readings

- Chapter 2 Section 2.2 and 2.4 of the «Parallel Programming Concept and Practice» book
- Designing and Building Parallel Program by Ian Foster (online) https://www.mcs.anl.gov/~itf/dbpp/

- To dive deeper into the world of interconnection networks and routing:
  - «Principles and Practices of Interconnection Networks» by W. Dally and B. Towles, Morgan Kaufmann, 2004
  - «Interconnection Networks – An Engineering Approach » by J. Duato et al., Morgan Kaufmann, 2022